How the bps firmware works

bps (bird positioning system) microphone nodes · `firmware/` at commit `da40411` (“Batch mode”) · kanban #7

Each node records audio, keeps only the stretches where a noise gate hears something, stamps every sample on the server's clock, and sends those chunks to `bsp-server`, which runs BirdNET and estimates direction from the arrival times across nodes. The firmware is a **portable C core** (`lib/bps_core/src/`) plus a thin **port layer** per board (`src/port/esp32.cpp`, `rp2.cpp`, shared `sockets.c`) behind `port.h`. `src/main.cpp` builds a `bps_settings` from compile-time defines (`bps.conf` → `scripts/bps_config.py` → `board_config.h`), calls `bps_client_setup()` once and `bps_client_loop()` from Arduino's `loop()`. The same core runs on Linux as `host/bps-host-client` (WAV replay, simulated card and radio) and is unit-tested in `test/` (`pio test -e native`). Boards: Pico W / Pico 2 W, ESP32, XIAO ESP32-S3, and the custom `bps_xiao` PCB (XIAO S3, PDM mic, MAX17048 gauge, microSD).

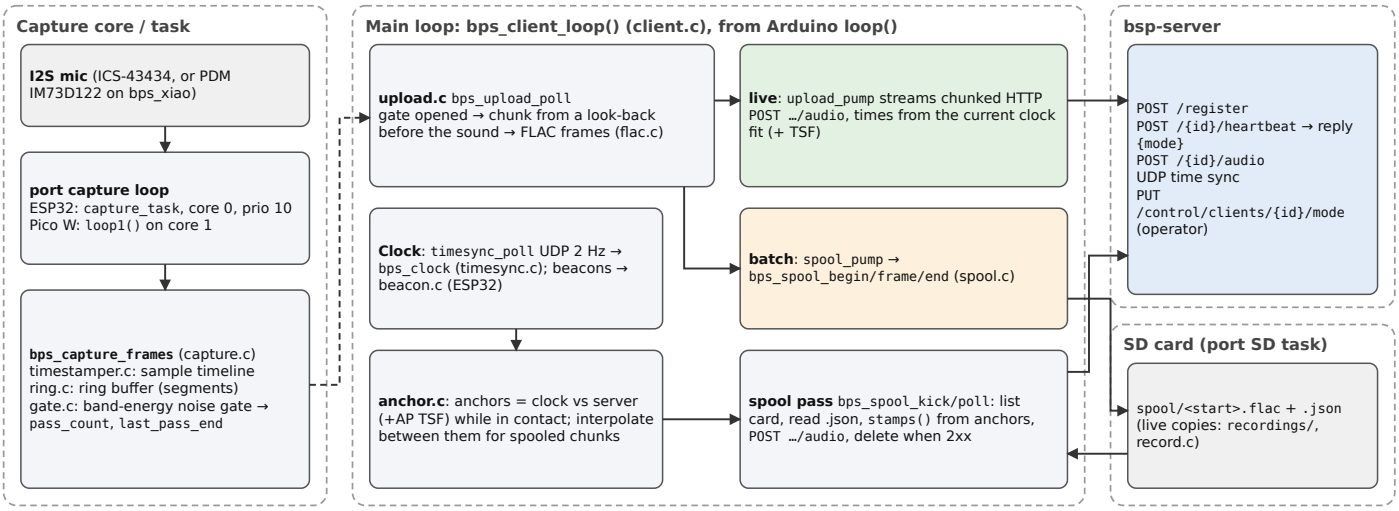


Figure 1. Architecture. Capture runs on its own core/task and only writes the ring and gate fields; everything else runs single-threaded and non-blocking in `bps_client_loop()`. Dashed: the ring and gate fields, read lock-free. Green/orange: where a chunk goes in live/batch mode. Card I/O goes through the port's SD task queue (`port_sd_*`).

Capture and timestamps

- **Capture side.** The port reads one DMA buffer at a time and calls `bps_capture_frames()` (capture.c:82). On ESP32 that is `capture_task` pinned to core 0 at priority 10 (esp32.cpp:407, :502), stamped from the I2S interrupt; on the Pico W it is `loop1()` on core 1 (rp2.cpp:234). `process()` (capture.c:38) takes the mic channel's top 16 bits, feeds the per-block capture time to the timestamper (a line fit giving each sample's monotonic time; a jump >0.5 ms starts a new ring segment and counts a dropout), writes the ring, and runs the gate, which publishes `pass_count`/`last_pass_end`.
- **Chunks.** When idle, `bps_upload_poll()` (upload.c:433) waits for the first clock fit and a new gate pass, then `upload_begin()` (upload.c:222) starts a chunk of `chunk_secs` (from registration) beginning a look-back before the sound, continuing gap-free if the gate stays open, never spanning a timeline jump. Its start/end are put on the server's clock by the current fit (`bps_clock_at`); with beacons, also on the access point's TSF. Samples overwritten before being read go out as silence.
- **Clock.** `timesync_poll()` (client.c:580) sends an NTP-style 36-byte UDP request every 0.5 s and spins up to 20 ms for the reply. The first reply sets `epoch_ns`; later ones feed the estimator (timesync.c). On ESP32, beacons of the joined AP (promiscuous RX, `port_beacon_take`) feed a TSF line (beacon.c); the Pico W's CYW43 has no beacon timestamps. Heartbeats report clock status and the AP-to-server mapping.
- **Anchors** (anchor.c). While in contact and measured afresh (`sync_poll`, client.c:684), `push_anchor()` (client.c:658) records local time vs server time (and TSF) every 10 s; only the first and latest per session are kept (64 total). Audio recorded with the radio off is later placed by interpolating between the anchors either side, which removes the drift over the gap.

How the firmware enters and switches modes

- **Not a build option or config flag.** No `bps.conf`/`platformio.ini` setting selects the mode. Every board **boots live**: `mode.run` is zero-initialised to `MODE_LIVE` (client.c:58, :153).
- **Server command.** The operator sets it with `PUT /api/v1/control/clients/{id}/mode {"mode":"batch","interval_s":300}` (bsp-server control.rs:166; 30–3600 s), stored in the database. The server returns it in the body of each heartbeat reply (client.rs:188). `heartbeat_reply()` (client.c:842) parses `mode`/`interval_s` (clamped to 30–3600) and calls `apply_mode()` (client.c:804). A batch client hears changes only at its next contact.
- **Runtime conditions** can keep or force live, reported back as `mode.note` in the heartbeat: batch needs a mounted card with ≥16 MB free (`card_has_room`, client.c:703); if the card goes or fills during a contact, `contact_poll` switches to live (client.c:753); in `ST_OFF` a missing card wakes the radio at once (client.c:1282); `go_off` stays live if `port_wifi_off()` fails (client.c:714). Pico boards always stay live: no card slot (`port_sd_begin` returns false, rp2.cpp:265) and `port_wifi_off` returns false (rp2.cpp:63).
- **Back to live:** a reply with `"live"` sets `to_spool` false and kicks a spool pass for what is left on the card (client.c:812–820); if that reply was a contact's final heartbeat, `go_off` is skipped (client.c:1023), so the radio stays on.

Live mode: the sending loop

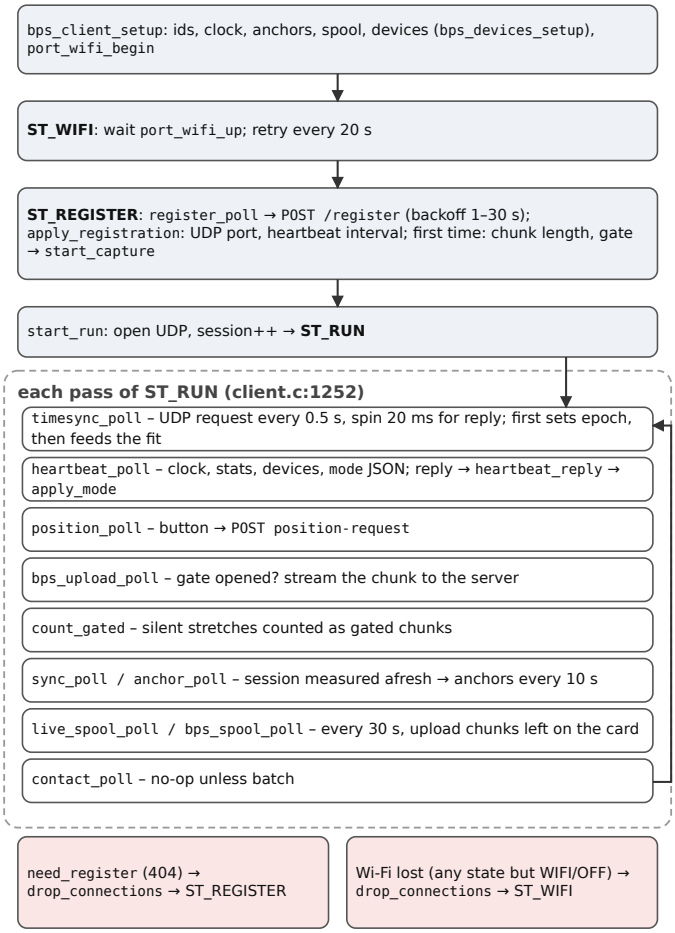


Figure 2. Live flow. States are `ST_WIFI/ST_REGISTER/ST_RUN/ST_OFF` (client.c:57); `ST_OFF` is only used in batch mode.

Every request is a non-blocking `txn` (one buffered HTTP POST, 10 s timeout; client.c:232–277), so one pass never waits long; the ESP32's `loop()` then `delay(1)`s (main.cpp:41). In live mode `to_spool` is false and `upload_pump()` (upload.c:380) streams the chunk as chunked HTTP (one FLAC frame per HTTP chunk, up to 16 writes per pass) to `POST /api/v1/client/{id}/audio` with `x-bsp-start-ns`/`x-bsp-end-ns` and, with beacons, `x-bps-tsf`. A stall of 5 s or a lost connection aborts it. If a card is present, `bps_record_*` (record.c) keeps a copy under `recordings/`. Chunks still on the card from batch mode are uploaded by `live_spool_poll()` (client.c:859) every 30 s once the session is synced.

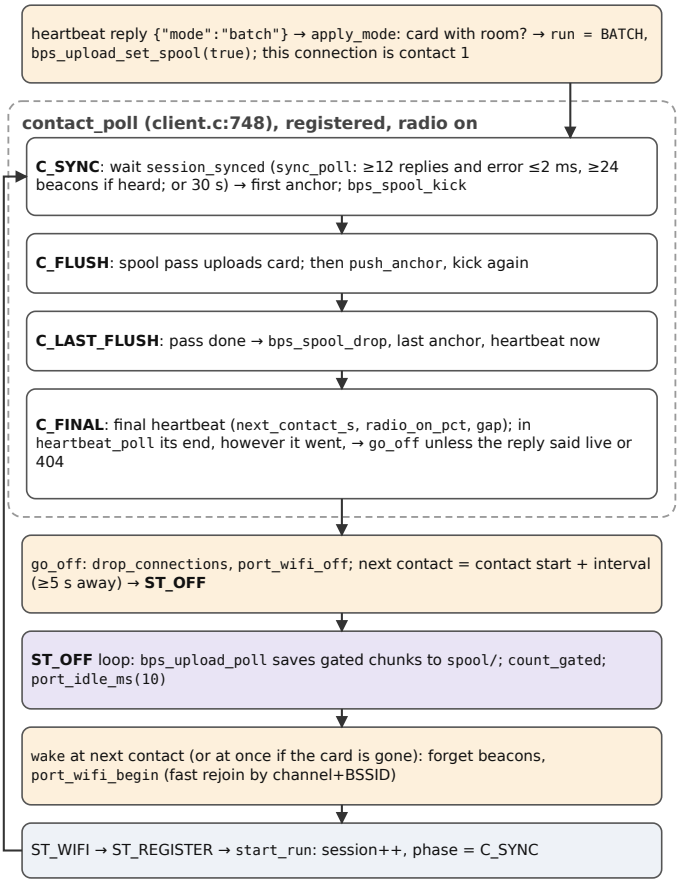


Figure 3. Batch flow. Contacts last at most 120 s (`CONTACT_MAX_US`); one that cannot register within 30 s goes back to sleep (`contact_connect_poll`, client.c:795).

Batch mode: radio off, audio on the SD card

With `bps_upload_set_spool(true)`, `upload_begin()` hands each chunk to `spool_begin()` (upload.c:197, :250) instead of the network. `spool_pump()` (upload.c:418) FLAC-encodes frames as audio arrives and writes them to `spool/<start_ms>.flac`; `bps_spool_end()` (spool.c:88) then writes `<start>.json` (local start/end, the fit's estimate, `epoch_ns`, size). This also happens in `ST_OFF`, where the loop runs only `bps_upload_poll`, `count_gated`, devices, LED and `port_idle_ms(10)` (client.c:1277–1290). A contact's spool pass (spool.c:434 `step()`) lists `*.json`, reads each, and `stamps()` (spool.c:282) interpolates server times (and TSF if both anchors follow the same AP without a clock jump) between anchors; a chunk with no anchor after it waits for the next pass, which is why `C_FLUSH` pushes an anchor and runs a second pass. Chunks from before a reboot (different `epoch_ns`) go up with their recorded estimate and no TSF. On 2xx both files are deleted; other 4xx deletes them too; 404 triggers re-registration; other failures leave them for later. Once per boot, orphaned `.flac` files are removed.